Wordle Game – C Programming Project

• A Terminal Implementation in C •

Coded and presented by Imad Bara and Alim Kamel

• Overview

The project detailed in this report is a terminal-based implementation of the popular word-guessing game, Wordle. Wordle is a simple yet engaging game where players have six attempts to guess a secret five-letter word, receiving color-coded feedback on their guesses. This feedback indicates whether a letter is in the correct position (green), in the word but in the wrong position (yellow), or not in the word at all (gray)

The primary objective of this project was to develop a fully functional version of Wordle using the C programming language, designed to run entirely within a command-line interface or terminal. This project serves as a practical application of fundamental C programming concepts, including string manipulation, array management, file I/O for word list handling, and conditional logic. The scope of this implementation includes a predefined list of valid words, a mechanism for random word selection for each game session, and the use of ANSI escape codes to provide the user with the classic color-coded feedback central to the Wordle experience.

• System Analysis & Requirements

1 . Game Rules : 6 Attempts to guess the correct word

Depending of the letter you typed :

COLOR	GREEN	YELLOW	GREY
RIGHT LETTER	✓	✓	✗
RIGHT SPOT	✓	✗	✗

3. System Design

This section outlines the logical structure and data organization used to implement the game and solver.

• Data Structures

To handle the dictionary, game state, and solver logic efficiently, we utilized specific data structures. The dictionary is stored as a dynamically allocated array of strings (char **dictionary). This allows the program to handle varying dictionary sizes without limits. For memory management, we use malloc to allocate the array pointer and strdup for each individual word. A corresponding free_dictionary function ensures no memory leaks occur when the program terminates.

We also defined an enumeration called Color in the header file to standardize the feedback mechanism (GREEN, YELLOW, GRAY), making the code readable and easier to maintain. The logic solver relies on several specialized arrays to deduce the correct word. A boolean array called used_words tracks which words have already been guessed to prevent repetition. Two 2D boolean arrays, correct_pos and wrong_pos, mark if a specific letter must be at a specific position or is forbidden at that position. Additionally, integer arrays min_count and max_count track the minimum and maximum possible occurrences of each letter to refine the search space with every guess.

Control Flow and Algorithms

The system employs specific algorithms for validation and game logic. The word validation function uses a linear search to ensure user input exists within the loaded dictionary before accepting it. The feedback computation operates in two distinct passes. The first pass checks for exact matches (Green) and counts the remaining letters in the target word. The second pass checks for displaced letters (Yellow) based on the remaining counts, defaulting to Gray if neither condition is met.

For the solver, we implemented an algorithm that calculates a score for every valid candidate word. It prioritizes words that contain the most unknown letters (where the minimum count is zero), which maximizes information gain in early turns and allows the solver to narrow down possibilities faster.

• Strategy Description

Word Selection Strategy

Our solver uses an Information Maximization method to optimize its guesses. Instead of selecting any valid random word from the dictionary, the solver calculates a specific score for every candidate word. The scoring logic increments a word's score by 1 for every letter in the word that has not yet been confirmed (where the minimum count is still zero). This strategy prioritizes exploration in the early game. By choosing words composed of letters that have not been tested yet, the solver aims to eliminate or confirm as many characters as possible in the first few turns, narrowing down the possibilities rapidly.

Eliminating Possibilities

After every guess, the feedback provided by the game (Green, Yellow, or Gray) is used to update four strict constraint tables. First, the correct_pos array tracks Green letters, marking that a specific letter must appear at a specific index. Second, the wrong_pos array tracks Yellow letters, marking that a specific letter cannot appear at that specific index. Third, the min_count array ensures the word must contain at least a certain number of instances of a letter. Finally, the max_count array ensures the word cannot contain more than a certain number of instances of a letter, which is derived from Gray letters. During the next pass, the solver iterates through the entire dictionary and strictly discards any word that violates even one of these four constraints.

Why is this approach effective?

This approach is effective because it reduces the search space, or the number of valid words with each turn. By strictly enforcing minimum and maximum letter counts, our solver can correctly handle complex cases like double letters (e.g., differeciating between SPEED and STARE), which is a common failure point in simpler algorithms. It ensures that the solver does not waste guesses testing letters it already has information about, leading to a more efficient solution path.

• Code Documentation

We used for this code multiple functions to simplify our task at finding a strategy to select guesses that efficiently narrow down possibilities and the most suitable data structure for this strategy.

Here's a list of all the functions me and imad used in this algorithm depending of their field and task :

1. Dictionary Management

load_dictionary(const char *filename)	Opens the .txt file, reads it line by line and loads valid 5 letter words into the dynamic memory, Returns the number of words loaded.
free_dictionary()	Frees the memory allocated for the dictionary array and the strings inside it to prevent memory leaks and overusage.

is_valid_word(const char *w)	Checks if a given word exists in the loaded dictionary using **strcmp**

2. Game Logic (ENGINE)

human_play()	Handles the mode where you play against the computer. It picks a random word (**target**) and gives you 6 tries to guess it using **print_feedback**.
solver_user_feedback()	Executes the automated solving algorithm and simulates game scenarios, It validates candidate candidates against the user-provided feedback.

• Complexity Analysis

To make this easier, we suggest the following :

N = Number of words in the dictionary (13992)

C = Word length (in this code 5)

P = Maximum guesses (in this code 6à)

Time Complexity

- Dictionary Loading : Converted to lowercase(5char)

$$O(N \cdot L) \Rightarrow O(N)$$

Time complexity : $O(N)=O(13992)$

- **Word Validation :** Scanning the dictionary

Worst case scenario : comparing with all the words

$$13992 \times 5 \approx 69960 \text{ comparaisons}$$

Time complexity : $O(N)$

- **Feedback Computation :** Two loops over 5 letters

$$2 \text{ X } 5 = 10$$

Time complexity : $O(1)$

- **Solver Filtering :** Iterate through all remaining words and simulate feedback.

$$6 \times 13992 \times 5 \approx 419760 \text{ Operations}$$

Time complexity $= O(G \cdot N \cdot L) \Rightarrow O(N)$

- Human Play Mode ： 6 guesses, each one validates againts all words

$$6 \times 13992 = 83952$$

Time complexity : O(N)

Space Complexity

- Dictionary Storage

  The size of the words.txt is 82KB

- Solver Memory

  Boolean array of 13992 + temporary array of 5

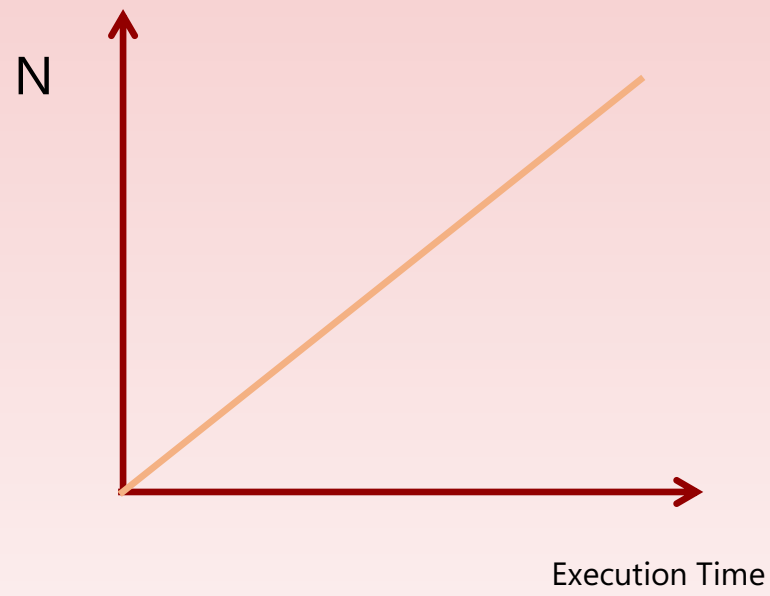
  Approximately 14KB

- Total Space Usage

The total space used is what we calculated so far in addition to the pointer array and many other variables, which makes it impossible to exactly know but for an approximation the full space used is under 250KB.

Graph Interpretation

We notice that the only variable that we interpreted before was the N (the number of words).



And we obtain a straight line meaning the Linear Complexity is verified

This project was presented and built by both Bara Imaddedine and Alim Kamel, thank you for your efforts during this semester teacher.